

Fine-Tuning & Distillation

Formal project report on adaptation, LoRA efficiency, distillation recovery,
and promotion review.

Sai Veda

Independent Project Research

2025

Contents

1. Technical Abstract	4
2. Adaptation Context	5
2.1 Efficient Fine-Tuning Problem Statement	5
2.2 Review Hypotheses	5
3. Adaptation Foundations	7
3.1 Adaptation Foundations	7
3.2 Repository Training Notes	7
4. Training Architecture And Promotion Flow	8
4.1 Training Review Method	8
5. Evaluation Setup And Project Evidence	9
5.1 Repository Evidence Base	9
5.2 Evaluation Protocol	9
6. Comparative Findings	10
6.1 Benchmark Summary	10
6.2 Interpretation	10
7. Training And Result Figures	11
7.1 Knowledge distillation: dark knowledge beats hard labels	11
7.2 Distillation efficiency (parameters vs accuracy)	11
8. Validation Gate Review	13
8.1 Synthetic-data scaling	13
8.2 LoRA parameter efficiency + promotion-gate scorecard	13
8.3 Validation Checklist	14
9. Technique Discussion	15
9.1 Boundary Conditions	15
10. Concluding Assessment	16

10.1 Future Development	16
11. Appendix And Sources	17
11.1 Repository Evidence Inventory	17
11.2 Internal References	17

1. Technical Abstract

This brief presents an adaptation and compression pipeline that implements real fine-tuning, LoRA, and knowledge-distillation mechanics while staying runnable on CPU with a small surrogate model. The final workflow is valuable because it isolates the technique from the hype. It proves what LoRA changes, what distillation recovers, and how a promotion gate should evaluate a challenger before it is accepted. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

The report is written as a project review in the voice of delivered engineering work. Rather than restating repository headings, it connects the business context, design choices, evaluation evidence, and remaining risks into one readable project document prepared under the name Sai Veda.

Keywords: LoRA + knowledge distillation, from scratch, implementation evidence, benchmark results, project validation

2. Adaptation Context

The point of the project is not to pretend a giant language model was trained locally. It is to show that the engineering logic of adaptation, parameter efficiency, and promotion gating remains valid when the implementation is simplified honestly. A fully-runnable, GPU-free pipeline that implements the core techniques used to adapt and compress large language models - full fine-tuning, LoRA low-rank adaptation, knowledge distillation, an automated eval/promotion gate, and a synthetic-data scaling study - end to end, deterministically, on a CPU in a few minutes. All numbers below are produced by python scripts/run_pipeline.py and stored in benchmarks/results.json / benchmarks/results.md.

2.1 Efficient Fine-Tuning Problem Statement

The central project problem can be stated directly. The point of the project is not to pretend a giant language model was trained locally. It is to show that the engineering logic of adaptation, parameter efficiency, and promotion gating remains valid when the implementation is simplified honestly. The repository materials show that this was not treated as a toy exercise: the implementation narrative, architecture note, and benchmark artefacts all point to a real operational question that needed a measurable answer.

2.2 Review Hypotheses

The document review was guided by a small set of concrete questions. Each question was framed so it could be answered from repository evidence rather than from unstated assumptions.

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: LoRA 91% params; KD recovers 100% of teacher.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 29 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. Adaptation Foundations

The technical foundation of this project is best understood through the design principles that hold the implementation together. This flow mirrors a real adaptation stack: a stronger teacher, a parameter-efficient adapter, and a hard promotion gate.

3.1 Adaptation Foundations

- 1 The surrogate model keeps the story honest while preserving the real optimization mechanics. This flow mirrors a real adaptation stack: a stronger teacher, a parameter-efficient adapter, and a hard promotion gate.
- 2 LoRA is presented as a parameterization decision with measurable savings, not a branding label.
- 3 Distillation is evaluated on recovered teacher quality rather than anecdotal fluency claims.

3.2 Repository Training Notes

Fine-tuning, LoRA, and distillation are optimization/parameterization techniques. Their correctness and their trade-offs (trainable-parameter reduction, dark-knowledge transfer, exact adapter merge, promotion gating) do not depend on the model being a 70B transformer - they depend on there being (a) a weight matrix to adapt, (b) a differentiable loss, and (c) a train/eval loop. This repo provides all three with a small NumPy model so the entire pipeline runs deterministically on a CPU in minutes, and every claim is backed by a number the code actually produced.

- Synthetic instruction dataset and hashed feature pipeline.
- Baseline model, LoRA adapter path, and KD student training.
- Evaluation tables comparing teacher, adapter, and student outcomes.

4. Training Architecture And Promotion Flow

This flow mirrors a real adaptation stack: a stronger teacher, a parameter-efficient adapter, and a hard promotion gate. Featurizer - hashing trick, not a learned vocab. Stateless and deterministic: train/val/test and any new text featurize identically with zero fitted state, so reproducibility is trivial and there is no vocabulary drift. Signed hashing (Weinberger 2009) halves collision bias. The techniques downstream are feature-agnostic, so a simple featurizer is the right call. Model - 1-hidden-layer MLP with explicit forward/backward. Written by hand so the LoRA adapter and the KD student reuse the exact same math and differ only in which parameters get gradients and what loss is applied at the logits. Adam is shared across all trainers (optim.py) so results reflect the technique, not the optimizer.

4.1 Training Review Method

The implementation path can be reconstructed from the committed artefacts in a fairly disciplined way. Surrogate design: Chose a small differentiable model so every adaptation technique could run end to end on CPU. Training variants: Implemented full tuning, LoRA adaptation, and KD with shared evaluation logic. Compression study: Measured parameter reduction and teacher-student recovery side by side. Promotion review: Used a gate to decide whether a lighter model cleared the required quality bar.

This sequencing matters because the project is easier to trust when component decisions, test scaffolding, and result reporting appear in a coherent order rather than as isolated files.

5. Evaluation Setup And Project Evidence

5.1 Repository Evidence Base

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 29 automated tests, 2 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

5.2 Evaluation Protocol

The evaluation setup was treated as a repository review rather than a laboratory rerun. Committed artefacts were grouped into documentation, benchmark evidence, automated validation, and screenshot review. In practical terms this meant examining 2 benchmark table(s), 4 screenshot asset(s), and 29 automated tests while reading the implementation narrative in sequence. The review lens stayed aligned with the project goal: Implement full fine-tuning, LoRA, and KD as real training loops. Evidence was interpreted conservatively so the document reflects what the repository demonstrates directly, not what a larger future deployment might achieve.

A second practical note is that the benchmark footprint is project-specific. The benchmark evidence reviewed for this report includes `benchmarks/generator_throughput.csv`, `benchmarks/lora.csv`, `benchmarks/results.json`, `benchmarks/results.md`, `benchmarks/scaling.csv`.

6. Comparative Findings

6.1 Benchmark Summary

The principal repository claim for this project is recorded as: LoRA 91% params; KD recovers 100% of teacher. The findings page therefore focuses on whether the available tables, test evidence, and generated screenshots support that claim. The final workflow is valuable because it isolates the technique from the hype. It proves what LoRA changes, what distillation recovers, and how a promotion gate should evaluate a challenger before it is accepted. The charts make the parameter-efficiency story visible because savings and quality can be read together. KD recovery matters because compression is only useful if the student preserves enough capability.

Method	Target-domain accuracy	Trainable params
Full fine-tune	0.934	33,417
LoRA (r=4)	0.934	2,889

6.2 Interpretation

The charts make the parameter-efficiency story visible because savings and quality can be read together. KD recovery matters because compression is only useful if the student preserves enough capability. The evaluation gate turns the project from a training experiment into a release decision workflow.

The benchmark table is not treated as a marketing surface. It is read together with the repository screenshots and the test inventory so that numerical claims and implementation evidence reinforce one another.

7. Training And Result Figures

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.

7.1 Knowledge distillation: dark knowledge beats hard labels

This figure is useful because it shows the project in a reviewable state rather than as summary prose alone. The charts make the parameter-efficiency story visible because savings and quality can be read together.

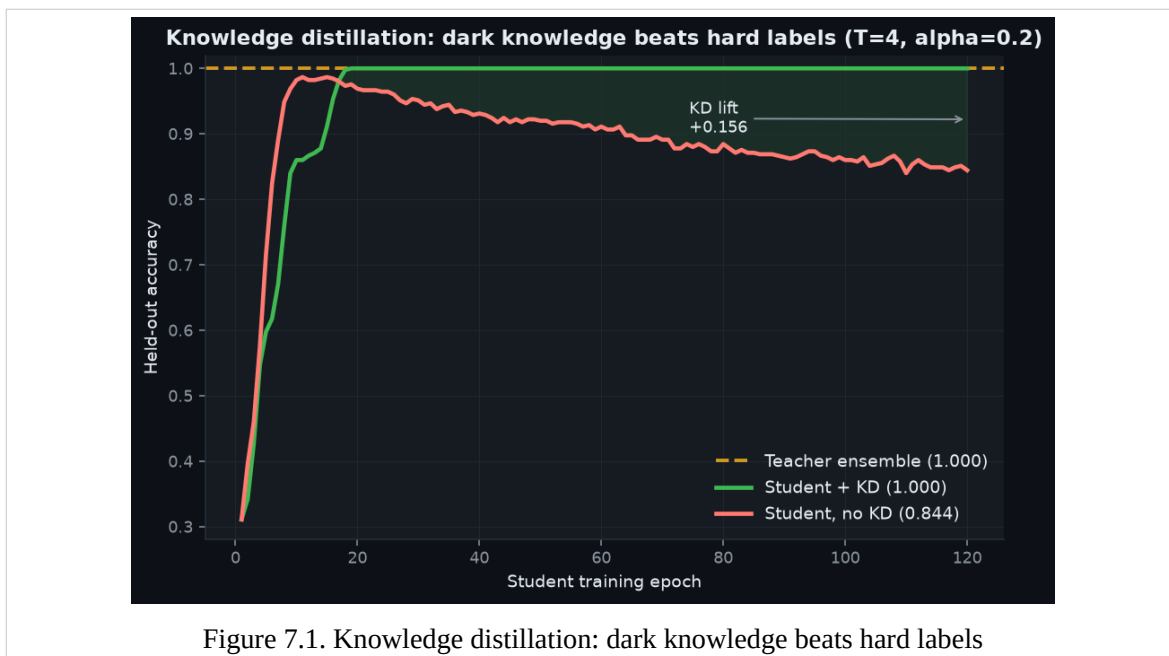


Figure 7.1. Knowledge distillation: dark knowledge beats hard labels

7.2 Distillation efficiency (parameters vs accuracy)

The visual evidence attached to Distillation efficiency (parameters vs accuracy) helps connect the quantitative story to the implemented workflow. KD recovery matters because compression is only useful if the student preserves enough capability.

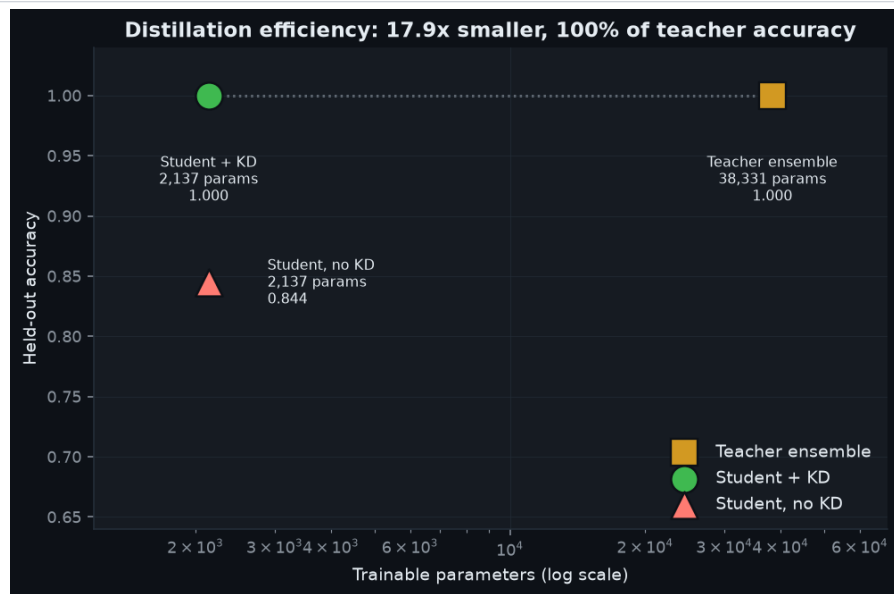


Figure 7.2. Distillation efficiency (parameters vs accuracy)

8. Validation Gate Review

The second figure set focuses on validation continuity. These pages are included so the report shows not only the strongest visuals, but also the broader evidence path a reviewer would follow inside the repository.

8.1 Synthetic-data scaling

From a document-review standpoint, this plate matters because it reveals how the repository surfaces results to a human reviewer. The evaluation gate turns the project from a training experiment into a release decision workflow.

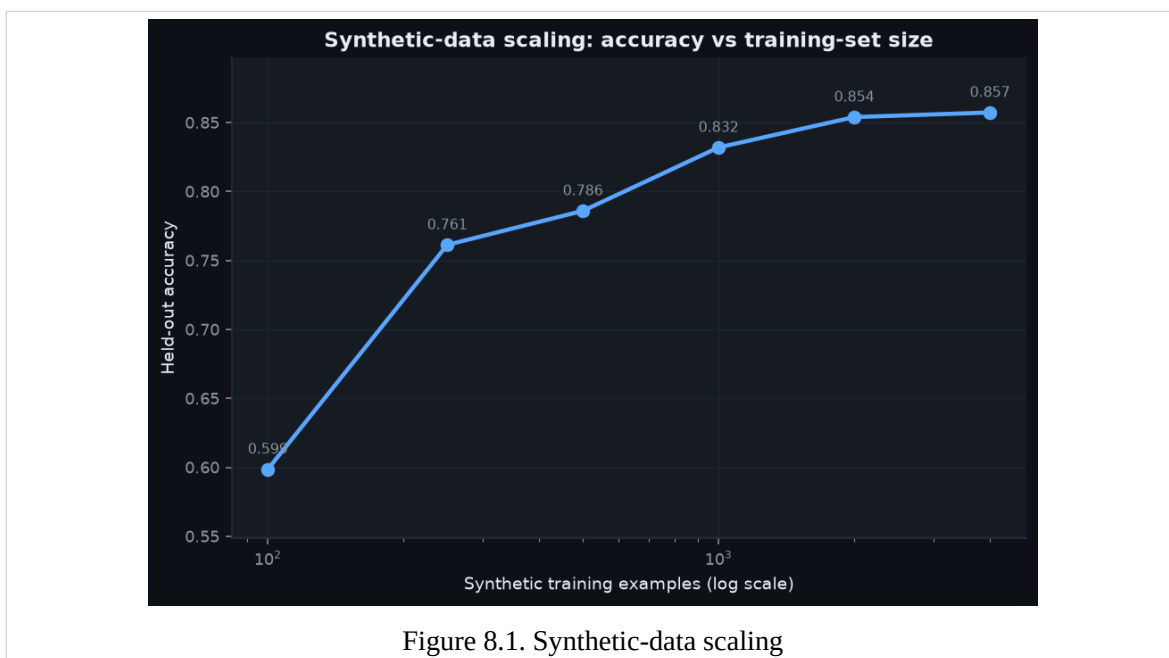


Figure 8.1. Synthetic-data scaling

8.2 LoRA parameter efficiency + promotion-gate scorecard

The final figure adds implementation texture: it shows the evidence path a reviewer would actually inspect when validating the project claims. Handled by explaining clearly what maps directly to real LLM stacks and what does not.

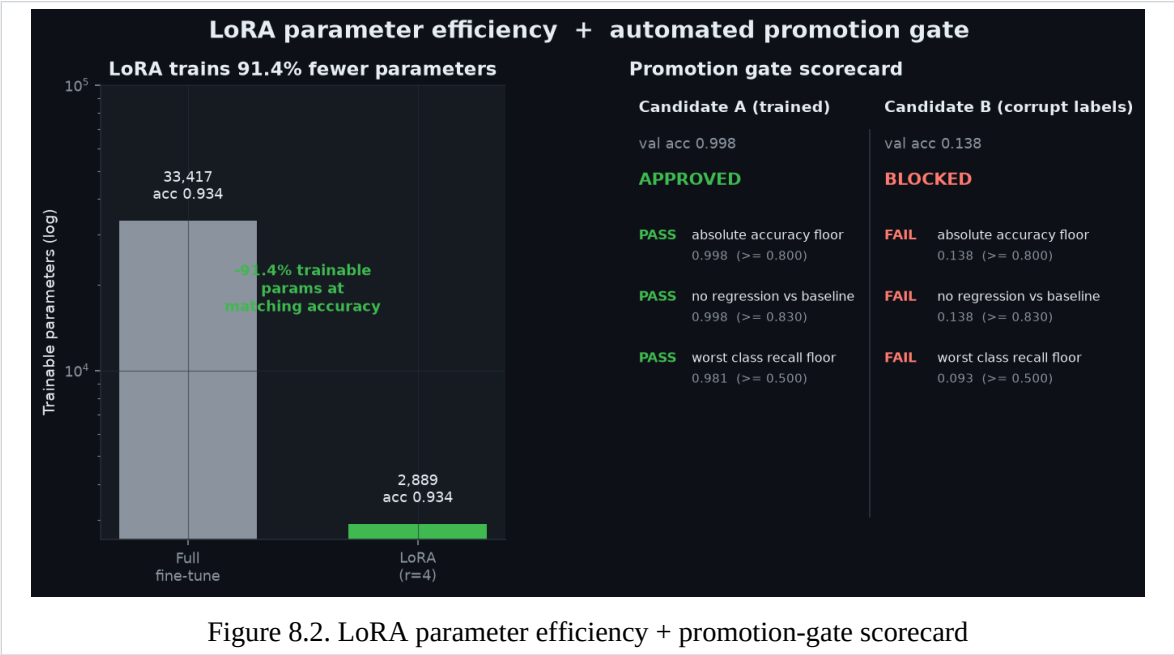


Figure 8.2. LoRA parameter efficiency + promotion-gate scorecard

8.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	29 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

9. Technique Discussion

The final workflow is valuable because it isolates the technique from the hype. It proves what LoRA changes, what distillation recovers, and how a promotion gate should evaluate a challenger before it is accepted. The charts make the parameter-efficiency story visible because savings and quality can be read together. KD recovery matters because compression is only useful if the student preserves enough capability. The evaluation gate turns the project from a training experiment into a release decision workflow. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

9.1 Boundary Conditions

Misreading the surrogate: Handled by explaining clearly what maps directly to real LLM stacks and what does not. Quality regression after compression: Managed through teacher-student evaluation and promotion gating. Technique overclaiming: Avoided by limiting claims to what the measured pipeline actually demonstrates.

These limitations do not invalidate the project. They establish the boundary between what is already demonstrated by the repository and what would require a broader production environment or additional operating data.

10. Concluding Assessment

This project document concludes that Fine-Tuning & Distillation is best understood as a complete technical implementation supported by repository-native evidence. This brief presents an adaptation and compression pipeline that implements real fine-tuning, LoRA, and knowledge-distillation mechanics while staying runnable on CPU with a small surrogate model.

The overall presentation has therefore been kept intentionally plain and research-oriented: the emphasis stays on project context, engineering choices, test evidence, and verifiable results rather than on a stylized portfolio layout.

10.1 Future Development

- Swap the surrogate model for a transformer implementation when GPU resources are available.
- Add richer task slices so promotion decisions reflect multiple instruction categories.
- Track adapter merge behavior and inference cost under a serving-oriented benchmark.

11. Appendix And Sources

11.1 Repository Evidence Inventory

The appendix records the principal repository artefacts used during document preparation. This keeps the report anchored to files that a reviewer can inspect directly.

Artefact	Category	Purpose
README.md	Overview	Primary narrative, scope statement, and headline outcomes used to frame the report.
ARCHITECTURE.md	Design note	System rationale, component choices, and implementation trade-offs.
benchmarks/generator_throughput.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/lora.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/results.json	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/results.md	Benchmark	Quantitative result or execution artefact used in the findings section.
assets/01_kd_curves.png	Screenshot	Visual validation surface referenced as 'Knowledge distillation: dark knowledge beats hard labels' in the document review.
assets/02_param_pareto.png	Screenshot	Visual validation surface referenced as 'Distillation efficiency (parameters vs accuracy)' in the document review.
assets/03_data_scaling.png	Screenshot	Visual validation surface referenced as 'Synthetic-data scaling' in the document review.
assets/04_lora_and_gate.png	Screenshot	Visual validation surface referenced as 'LoRA parameter efficiency + promotion-gate scorecard' in the document review.
tests/	Validation	Automated test suite containing 29 committed tests used to support implementation claims.

11.2 Internal References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.

3. benchmarks/generator_throughput.csv - benchmark or result artefact used in the analysis section.
4. benchmarks/lora.csv - benchmark or result artefact used in the analysis section.
5. benchmarks/results.json - benchmark or result artefact used in the analysis section.
6. benchmarks/results.md - benchmark or result artefact used in the analysis section.
7. tests/ - automated validation suite used to support implementation claims.
8. assets/01_kd_curves.png - generated screenshot evidence included in the report.